



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Facultad de Computación

Base de Datos II (CS2042)

Proyecto 1: SIMULADOR DE GESTOR DE BASE DE DATOS CON ORGANIZACIÓN EFICIENTE DE ARCHIVOS

Integrantes

Sebastian Antonio Hernandez Miñano
Andrés Martín Benjamín Torres Ramos
David Andres Huette Ospino
Carlos Enrique Angel Oriundo
André Alejandro Contreras Valera

Profesor

Dr. Heider Sánchez

Lima, Perú - 2026-1

Índice

1. Introducción	2
1.1. Objetivos	2
1.1.1. Objetivo General	2
1.1.2. Objetivos Específicos	2
2. Arquitectura de Indexación	2
2.1. Sequential File (Archivo Secuencial)	2
2.2. Extendible Hashing (Hash Extensible)	3
2.3. B+ Tree	3
2.4. R-Tree (Árbol Espacial)	3
3. Administración de Páginas, Archivos de Tabla y Motor	4
3.1. PageManager	4
3.2. Table File Management	4
3.3. Engine	4
4. Parser SQL	5
4.1. Analizador Léxico (Scanner)	5
4.2. 4.2 Analizador Sintáctico (Parser)	6
4.2.1. Gramática Formal (EBNF)	6
4.2.2. Autómatas del Analizador Léxico	7
4.2.3. Descripción de Estados	7
4.2.4. Tratamiento de Tipos Complejos	7
5. Backend y Frontend	8
5.1. Backend	8
5.2. Frontend	8
6. Pruebas y Análisis Experimentales	9
6.1. Datasets Utilizados	9
6.1.1. Pokemon Complete 2025	9
6.1.2. Customers 10000	10
6.1.3. Customers 100000	10
6.1.4. Airbnb Locations	10
6.2. Consultas Representativas	11
6.3. Espacio para Comparación Experimental	11
6.4. Discusión y Análisis de Resultados	13
7. Conclusiones	13
8. Anexos	14
8.1. Repositorio del proyecto	14
8.2. Video de demostración	14
8.3. Evidencia de ejecución	14

1. Introducción

En el ámbito de las bases de datos, el rendimiento de un sistema está fuertemente condicionado por la forma en que interactúa con el disco. Debido a la latencia inherente de los dispositivos de almacenamiento, es imperativo que las estructuras de datos minimicen la cantidad de accesos físicos. Este proyecto implementa un sistema gestor de bases de datos (SGBD) orientado a la gestión eficiente de memoria secundaria.

A diferencia de soluciones simplificadas que operan sobre memoria principal, este sistema opera estrictamente a nivel de página (bloques físicos de 4 KB), cumpliendo con la restricción fundamental de no cargar archivos completos en la memoria RAM. Para lograrlo, se ha construido el motor desde cero, integrando un analizador sintáctico (Parser SQL) propio, un administrador de páginas a bajo nivel, y cuatro estructuras de indexación avanzadas que permiten resolver diversos tipos de consultas (exactas, de rango y espaciales) de manera óptima.

1.1. Objetivos

1.1.1. Objetivo General

Desarrollar un simulador funcional de un Sistema Gestor de Base de Datos (SGBD) basado en disco, capaz de procesar sentencias SQL y administrar la memoria secundaria de forma eficiente mediante paginación estricta y estructuras de indexación avanzadas.

1.1.2. Objetivos Específicos

- **Gestión de Memoria Secundaria:** Implementar un *PageManager* que administre la lectura y escritura de bloques de 4 KB, garantizando que los archivos de tabla (*Heap Files*) y los índices operen sin cargar la totalidad de los datos en RAM.
- **Procesamiento de Consultas:** Construir un Analizador Léxico y Sintáctico (Parser) basado en Autómatas Finitos y Análisis Descendente Recursivo para traducir consultas SQL en comandos tipados.
- **Implementación de Índices Estándar y Dinámicos:** Desarrollar un *Sequential File* para búsquedas ordenadas y un *Extendible Hashing* para búsquedas exactas con costo $O(1)$.
- **Implementación de Índices Arbóreos:** Construir un Árbol B+ para optimizar las consultas por rango (*BETWEEN*) y un R-Tree multidimensional para procesar consultas espaciales (*RADIUS* y *kNN*).
- **Evaluación Experimental:** Comparar empíricamente el rendimiento de cada estructura de indexación midiendo el tiempo de ejecución (ms) y la cantidad de accesos a disco (lecturas y escrituras) utilizando diversos datasets.

2. Arquitectura de Indexación

A continuación se detallan las cuatro estructuras de indexación implementadas, todas integradas con el sistema de paginación fija del *PageManager*.

2.1. Sequential File (Archivo Secuencial)

Implementado en `sequential.py`, esta estructura organiza los registros de forma ordenada por clave para optimizar búsquedas binarias.

- **Estructura:** Se divide en dos secciones: una principal ordenada físicamente y un área auxiliar (*overflow*) para nuevas inserciones.
- **Búsqueda:** Combina búsqueda binaria en la sección principal $O(\log N)$ con un escaneo lineal en la sección de desbordamiento $O(K)$.
- **Mantenimiento:** Cuando el área auxiliar alcanza un umbral K , se dispara una reconstrucción física que fusiona ambas secciones en un nuevo archivo ordenado, garantizando que el costo de búsqueda no degrade con el tiempo.

2.2. Extendible Hashing (Hash Extensible)

Implementado en `extendible_hash.py`, es un esquema de indexación dinámica para búsquedas exactas ultra-rápidas $O(1)$.

- **Directorio:** Mantiene un directorio de punteros a buckets. El tamaño del directorio es 2^D , donde D es la profundidad global.
- **Buckets:** Cada bucket tiene una profundidad local $d \leq D$. Cuando un bucket se llena, se divide en dos y se actualizan los punteros del directorio.
- **Dinamicidad:** Si $d = D$ al momento de dividir, el directorio se duplica automáticamente. No soporta búsquedas por rango, optimizando exclusivamente el acceso puntual.

2.3. B+ Tree

Implementado en `BPlusTree.py`, es la estructura estándar para gestión de rangos en SGBDs.

- **Nodos de Tamaño Fijo:** Los nodos internos y hojas ocupan exactamente una página física. El grado B está calculado para maximizar el uso del bloque de 4 KB.
- **Hojas Enlazadas:** Las hojas mantienen un puntero al siguiente hermano, permitiendo que un `rangeSearch` sea $O(\log N + M)$, donde M es la cantidad de páginas en el rango.
- **Balanceo:** Implementa algoritmos de *split* (división) y *merge* (fusión) para mantener el árbol balanceado con una ocupación mínima del 50 %.

2.4. R-Tree (Árbol Espacial)

Implementado en `Rtree.py`, permite indexar datos multidimensionales (puntos geográficos).

- **MBR (Minimum Bounding Rectangles):** Cada nodo agrupa sus hijos en rectángulos mínimos envolventes. Las consultas se realizan mediante pruebas de intersección entre el MBR y el área de búsqueda.
- **Consultas:**
 - **Radius Search:** Retorna puntos dentro de una distancia Euclidiana mediante poda de ramas que no intersecan con el círculo de consulta.
 - **kNN (k-Nearest Neighbors):** Implementa el algoritmo de *Best-First Search* usando una cola de prioridad para explorar los nodos más cercanos al punto objetivo.

3. Administración de Páginas, Archivos de Tabla y Motor

3.1. PageManager

El **PageManager**, implementado en `DB_source/page_manager.py`, es la capa de acceso físico a disco. Su objetivo es que el resto del sistema no lea ni escriba archivos completos, sino fragmentos ubicados dentro de páginas de tamaño fijo.

- **Paginación fija:** El archivo se divide en páginas de 4096 bytes. Toda lectura o escritura se traduce a una o más operaciones sobre páginas.
- **Lectura paginada:** Cuando un componente solicita bytes desde un offset, el **PageManager** calcula qué página contiene ese offset y lee únicamente el rango necesario.
- **Escritura paginada:** Para modificar bytes, se ubica la página física correspondiente y se escribe en la posición exacta sin reconstruir el archivo completo.
- **Métricas de E/S:** La capa registra contadores de lecturas y escrituras a disco, usados por el backend para reportar `disk_reads` y `disk_writes`.

Esta capa es importante para cumplir la restricción de la rúbrica: no se usa un motor de BD externo y el acceso a disco se realiza de forma paginada. Los índices y archivos de tabla dependen de esta abstracción para operar sobre offsets físicos.

3.2. Table File Management

El módulo `DB_source/Table_file_managment.py` administra el archivo principal de cada tabla. Este archivo funciona como un *heap file*: los registros se almacenan físicamente en el orden en que son insertados, y los índices guardan punteros hacia sus offsets.

- **Cabecera:** Al crear una tabla se escribe una cabecera con el formato físico del registro, cantidad de registros y banderas de índices.
- **Inserción:** `insert_record` serializa la tupla según el formato `struct` de la tabla, la escribe al final del archivo y retorna el offset físico.
- **Lectura por offset:** `read_record_at` permite recuperar un registro específico usando el puntero almacenado por un índice.
- **Iteración paginada:** `iter_records` recorre los registros desde disco sin cargar toda la tabla en memoria.
- **Eliminación lógica:** `delete_record` marca registros como eliminados usando una bandera, evitando reescrituras físicas costosas.

La decisión de usar un heap file separa almacenamiento e indexación: el archivo de tabla conserva los datos reales y cada índice mantiene su propio archivo con claves y offsets. Por ello, un registro puede ser encontrado por Hash, Sequential, B+ Tree o R-Tree sin duplicar la fila completa dentro del índice.

3.3. Engine

El **Engine**, implementado en `engine/engine.py`, conecta el parser SQL, el archivo de tabla y las estructuras de indexación.

- **Catálogo:** Mantiene metadata persistente de tablas, columnas, tipos e índices en `catalog.json`.

- **CREATE TABLE:** Construye el formato físico de cada registro, crea el heap file y registra qué columnas tienen índices.
- **Carga desde CSV:** Si la sentencia incluye **FROM FILE**, lee el CSV fila por fila, normaliza tipos y escribe registros en disco.
- **INSERT:** Inserta el registro en el heap file, obtiene el offset físico y actualiza cada índice asociado.
- **SELECT:** Usa el índice adecuado cuando existe. Para igualdad puede usar Hash, Sequential o B+ Tree; para rangos usa Sequential o B+ Tree; para consultas espaciales usa R-Tree.
- **DELETE:** Localiza los offsets mediante el índice, marca el registro como eliminado y actualiza las entradas del índice correspondiente.

El flujo general es:

1. El usuario envía una consulta SQL.
2. El parser convierte el texto en un comando estructurado.
3. El engine valida tabla, columnas y tipos.
4. El heap file almacena o recupera registros mediante offsets.
5. El índice elegido reduce el número de páginas consultadas.
6. El resultado se devuelve junto con métricas de tiempo y accesos a disco.

4. Parser SQL

4.1. Analizador Léxico (Scanner)

El **Scanner** implementa un autómata finito determinista (DFA) para convertir el flujo de caracteres en un flujo de tokens significativos.

- **Mecanismo:** Escaneo de un solo paso (*single-pass*) con punteros **first** y **current**.
- **Lógica de Autómatas:**
 - **Identificadores:** $[a - zA - Z_][a - zA - Z0 - 9_]*$. Se contrastan contra una tabla de símbolos de palabras reservadas (SQL Keywords).
 - **Números:** Soporte para enteros, negativos y decimales (puntos flotantes) necesarios para coordenadas espaciales.
 - **Cadenas:** Soporte para literales entre comillas simples o dobles.
 - **Símbolos Compuestos:** Reconocimiento de operadores multicarácter como $\leq$ y $\geq$ mediante *lookahead* de un carácter (**peek**).

4.2. 4.2 Analizador Sintáctico (Parser)

El **SQLParser** está compuesto por un **Scanner** (Analizador Léxico) y un **Parser** (Analizador Sintáctico). La implementación se basa en la técnica de **Análisis Sintáctico Descendente Recursivo** (Recursive Descent Parsing), la cual valida la gramática formal y traduce sentencias de un subconjunto restringido de SQL en una estructura de datos tipada denominada Árbol de Sintaxis Abstracta (AST).

El flujo de procesamiento consiste en:

1. **Tokenización:** El Scanner lee el flujo de caracteres y los agrupa en unidades con significado léxico (tokens).
2. **Validación Sintáctica:** El Parser consume los tokens siguiendo las reglas de una gramática formal.
3. **Generación de AST:** Se instancian objetos de tipo *Command* (utilizando *dataclasses* de Python) que representan la operación a ejecutar.

4.2.1. Gramática Formal (EBNF)

A continuación, se define la gramática formal empleada, escrita en notación EBNF (Extended Backus-Naur Form). Esta gramática cubre las operaciones de creación de tablas, selección, inserción y eliminación.

Listing 1: Gramática EBNF del Parser SQL

```
<program> ::= { <statement> }
<statement> ::= ( <create_stmt> | <select_stmt> |
                  <insert_stmt> | <delete_stmt> ) ";"

/* --- CREATE_STM --- */
<create_stmt> ::= "CREATE" "TABLE" <id> "(" <column_list> ")"
                [ "FROM" "FILE" <string> ]
<column_list> ::= <column_def> { "," <column_def> }
<column_def> ::= <id> <type_def> [ "INDEX" <index_tech> ]
<type_def> ::= <id> { <id> | <num> | "(" | ")" }
<index_tech> ::= "Sequential" | "Extendible" | "BPlus" | "RTree" | <id>

/* --- SELECT_STM --- */
<select_stmt> ::= "SELECT" "*" "FROM" <id> [ "WHERE" <condition> ]
<condition> ::= <point_condition> | <range_condition> |
                <spatial_condition>

<point_condition> ::= <id> <operator> <value>
<range_condition> ::= <id> "BETWEEN" <value> "AND" <value>
<spatial_condition> ::= <id> "IN" "(" "POINT" "(" <num> "," <num> ")"
                    "," <spatial_param> ")"

<spatial_param> ::= "RADIUS" <num> | "K" <num>
<operator> ::= "=" | "<" | ">" | "<=" | ">="

/* --- INSERT_STM --- */
<insert_stmt> ::= "INSERT" "INTO" <id> "VALUES" "(" <value_list> ")"
<value_list> ::= <value> { "," <value> }

/* --- DELETE_STM --- */
<delete_stmt> ::= "DELETE" "FROM" <id> "WHERE" <id> "=" <value>
```

```

/* --- TERMINALES GENERALES --- */
<id> ::= ID
<num> ::= NUM
<string> ::= STRING
<value> ::= <num> | <string>

```

4.2.2. Autómatas del Analizador Léxico

El Scanner funciona como un Autómata Finito Determinista (DFA) que transita entre estados según el carácter leído del flujo de entrada. A continuación se presenta el diagrama de estados simplificado para los tokens principales:

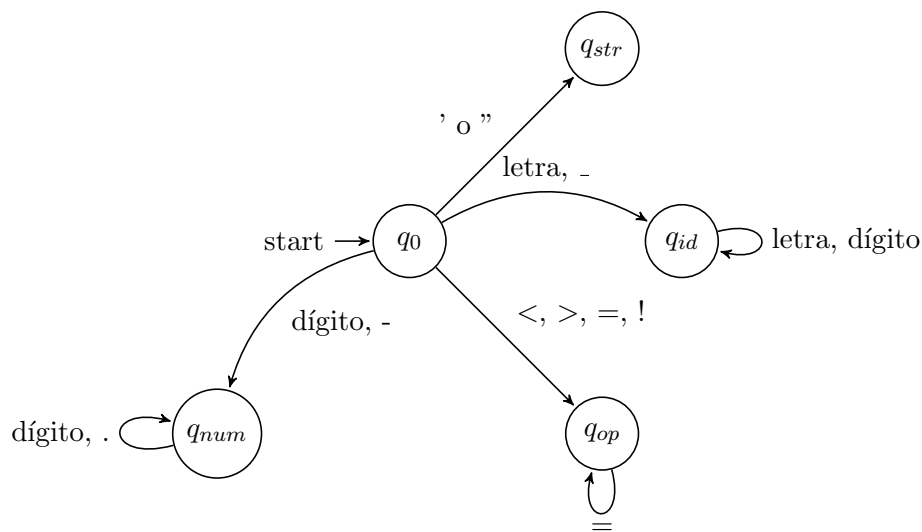


Figura 1: DFA simplificado del Scanner para identificación de tokens.

4.2.3. Descripción de Estados

- **q_0 (Inicio):** Estado inicial donde se descartan espacios en blanco y se decide el siguiente camino.
- **q_{id} (Identificador/Keyword):** Acumula caracteres alfanuméricos. Al finalizar, consulta una tabla de símbolos para determinar si es una palabra reservada (como **SELECT**) o un nombre de tabla.
- **q_{num} (Numérico):** Procesa números enteros y decimales. Soporta el signo negativo inicial para coordenadas en el R-Tree.
- **q_{str} (Cadenas):** Acumula caracteres hasta encontrar la comilla de cierre.

4.2.4. Tratamiento de Tipos Complejos

El parser implementa una lógica especial en `_parse_type_name()` para capturar definiciones de tipos que incluyen paréntesis (ej. `VARCHAR(50)`) mediante un contador de profundidad, asegurando que la definición de la columna sea capturada íntegramente antes de buscar la siguiente coma.

5. Backend y Frontend

5.1. Backend

El backend está implementado con `FastAPI` en `backend/app.py`. Su función es exponer el motor de base de datos mediante una API HTTP simple para que el frontend pueda enviar consultas SQL y mostrar resultados.

- **Inicialización:** Al iniciar el servidor, se crea una instancia global de `Engine` apuntando al directorio `runtime`.
- `GET /health`: Verifica que el servicio esté levantado.
- `GET /tables`: Devuelve la lista de tablas registradas en el catálogo del engine.
- `GET /tables/{table_name}`: Devuelve metadata de una tabla específica: columnas, índices, formato físico y cantidad de registros.
- `POST /query`: Recibe un JSON con una consulta SQL, llama a `engine.execute(sql)` y retorna filas, métricas y estado de ejecución.

La respuesta del backend incluye métricas experimentales generadas por el `PageManager`: tiempo de ejecución en milisegundos, lecturas a disco y escrituras a disco. Estas métricas son esenciales para comparar las técnicas de indexación.

5.2. Frontend

El frontend está implementado en `index.html`. Es una interfaz web liviana que permite enviar consultas SQL al backend y visualizar los resultados.

- **Entrada SQL:** El usuario escribe consultas como `CREATE TABLE`, `INSERT`, `SELECT` o `DELETE`.
- **Comunicación con backend:** El frontend realiza peticiones HTTP a `/query`.
- **Visualización de resultados:** Muestra filas devueltas por el motor, errores sintácticos o semánticos y estadísticas de ejecución.
- **Pruebas espaciales:** Permite ejecutar consultas de tipo `RADIUS` y `KNN` sobre columnas espaciales.

El flujo completo es:

1. El usuario ingresa una consulta en el navegador.
2. El frontend envía la consulta al backend mediante `POST /query`.
3. El backend invoca al engine.
4. El engine usa el parser, el heap file y el índice correspondiente.
5. El backend retorna filas y métricas.
6. El frontend renderiza la respuesta.

6. Pruebas y Análisis Experimentales

6.1. Datasets Utilizados

Para las pruebas experimentales se utilizan datasets de distintos dominios y tamaños, con índices elegidos según el tipo de consulta que se desea evaluar.

6.1.1. Pokemon Complete 2025

Dataset de Pokémon usado para validar búsquedas exactas por identificador, búsquedas secuenciales por nombre y consultas de rango por estadística total.

Listing 2: Tabla pokemon_complete_2025

```
CREATE TABLE pokemon_complete_2025 (
  pokedex_id INT INDEX HASH,
  name VARCHAR(50) INDEX SEQUENTIAL,
  genus VARCHAR(50),
  generation VARCHAR(10),
  type_1 VARCHAR(20),
  type_2 VARCHAR(20),
  num_types INT,
  hp INT,
  attack INT,
  defense INT,
  sp_attack INT,
  sp_defense INT,
  speed INT,
  base_stat_total INT INDEX BTREE,
  height_m FLOAT,
  weight_kg FLOAT,
  base_experience INT,
  ability_1 VARCHAR(50),
  ability_2 VARCHAR(50),
  hidden_ability VARCHAR(50),
  color VARCHAR(20),
  shape VARCHAR(30),
  habitat VARCHAR(30),
  growth_rate VARCHAR(30),
  egg_groups VARCHAR(50),
  is_legendary VARCHAR(10),
  is_mythical VARCHAR(10),
  is_baby VARCHAR(10),
  capture_rate INT,
  base_happiness INT,
  hatch_counter INT,
  gender_rate INT,
  description VARCHAR(255),
  sprite_url VARCHAR(150),
  is_dual_type VARCHAR(10),
  bmi FLOAT,
  attack_defense_ratio FLOAT,
  physical_total INT,
  special_total INT,
  offensive_total INT,
  defensive_total INT,
  gender_distribution VARCHAR(50),
  stat_tier VARCHAR(50)
```

```
);
```

6.1.2. Customers 10000

Dataset de clientes usado principalmente para probar *Sequential File*, overflow y reconstrucción física sobre `Customer_Id`. También incluye Hash sobre `id` y B+ Tree sobre `Subscription_Date`.

Listing 3: Tabla customers_10000

```
CREATE TABLE customers_10000 (  
  id INT INDEX HASH,  
  Customer_Id VARCHAR(20) INDEX SEQUENTIAL,  
  First_Name VARCHAR(60),  
  Last_Name VARCHAR(60),  
  Company VARCHAR(120),  
  City VARCHAR(80),  
  Country VARCHAR(80),  
  Phone_1 VARCHAR(40),  
  Phone_2 VARCHAR(40),  
  Email VARCHAR(120),  
  Subscription_Date VARCHAR(20) INDEX BTREE,  
  Website VARCHAR(160)  
);
```

6.1.3. Customers 100000

Dataset de clientes de mayor tamaño usado para probar *Extendible Hashing* sobre `First_Name`, además de búsquedas exactas por `id`, búsqueda secuencial por `Customer_Id` y rangos por fecha de suscripción.

Listing 4: Tabla customers_100000

```
CREATE TABLE customers_100000 (  
  id INT INDEX HASH,  
  Customer_Id VARCHAR(20) INDEX SEQUENTIAL,  
  First_Name VARCHAR(60) INDEX HASH,  
  Last_Name VARCHAR(60),  
  Company VARCHAR(120),  
  City VARCHAR(80),  
  Country VARCHAR(80),  
  Phone_1 VARCHAR(40),  
  Phone_2 VARCHAR(40),  
  Email VARCHAR(120),  
  Subscription_Date VARCHAR(20) INDEX BTREE,  
  Website VARCHAR(160)  
);
```

6.1.4. Airbnb Locations

Dataset espacial derivado de Airbnb New York. Se usa para probar R-Tree mediante consultas por radio y `k` vecinos más cercanos. También incluye Hash sobre `id` y B+ Tree sobre `price`.

Listing 5: Tabla airbnb_locations

```
CREATE TABLE airbnb_locations (  
  id INT INDEX HASH,
```

```

name VARCHAR(160),
host_id VARCHAR(40),
host_name VARCHAR(80),
neighbourhood_group VARCHAR(80),
neighbourhood VARCHAR(100),
coords POINT2D INDEX RTREE,
room_type VARCHAR(40),
price INT INDEX BTREE,
minimum_nights INT,
number_of_reviews INT,
last_review VARCHAR(20),
reviews_per_month FLOAT,
calculated_host_listings_count INT,
availability_365 INT
);

```

6.2. Consultas Representativas

Las consultas experimentales se agrupan por técnica de indexación:

- **Hash:** búsquedas exactas por `id`, `pokedex_id` y `First.Name`.
- **Sequential File:** búsquedas exactas y de rango por `Customer_Id` o `name`.
- **B+ Tree:** búsquedas de rango por `base_stat_total`, `Subscription_Date` y `price`.
- **R-Tree:** consultas `RADIUS` y `KNN` sobre `coords`.

6.3. Espacio para Comparación Experimental

La siguiente tabla se completará con los resultados obtenidos al ejecutar los casos de prueba. Para cada técnica de indexación se incluyen operaciones de inserción y búsqueda, de modo que se cubra explícitamente lo solicitado por la rúbrica. Se registrarán dos métricas exigidas: tiempo de ejecución en milisegundos y total de accesos a disco, separando lecturas y escrituras.

Cuadro 1: Comparación de desempeño de técnicas de indexación

Dataset	Índice	Operación	Caso de prueba	Reads	Writes	Tiempo (ms)
Pokemon	Hash	Inserción	Nuevo <code>pokedex_id</code>	23	3	958.637
Pokemon	Hash	Búsqueda	<code>pokedex_id = 25</code>	15	0	23.027
Pokemon	Sequential	Inserción	Nuevo <code>name</code>	23	2	920.431
Pokemon	Sequential	Búsqueda	<code>name = 'pikachu'</code>	15	0	22.000
Pokemon	B+ Tree	Inserción	Nuevo <code>base_stat_total</code>	23	2	47.221
Pokemon	B+ Tree	Búsqueda rango	<code>base_stat_total BETWEEN 500 AND 700</code>	2562	0	537.081
Customers 10000	Hash	Inserción	Nuevo cliente con <code>id = 200001</code>	23	2	3.746
Customers 10000	Hash	Búsqueda	<code>id = 1</code> (Heather Callahan)	15	0	3.047
Customers 10000	Sequential	Inserción/Rebuild	Clientes SeqTest001–SeqTest105 con claves B001–B105	69	7	6.035
Customers 10000	Sequential	Búsqueda	<code>Customer_Id = 'EB54EF1154C3A78'</code> (Heather)	15	0	1.900
Customers 10000	Sequential	Búsqueda post-rebuild	<code>Customer_Id = 'B050'</code>	7	0	0.903
Customers 10000	B+ Tree	Inserción	Nuevo cliente con fecha 2020-06-15	23	2	2.632
Customers 10000	B+ Tree	Búsqueda rango	<code>Subscription_Date BETWEEN '2020-01-01' AND '2020-12-31'</code>	34553	0	627.174
Customers 100000	Hash	Inserción/Split	Nuevos clientes con <code>First_Name = 'Sebastian'</code>	23	2	4.231
Customers 100000	Hash	Búsqueda	<code>First_Name = 'Jared'</code>	469	0	41.200
Customers 100000	Hash	Búsqueda post-split	<code>First_Name = 'Sebastian'</code>	130	0	3.564
Customers 100000	Sequential	Inserción	Nuevo cliente con <code>Customer_Id = 'SEQHASH002'</code>	23	2	3.002
Customers 100000	Sequential	Búsqueda	<code>Customer_Id = 'ffeCAb7AbcB0f07'</code> (Jared)	15	0	1.998
Customers 100000	Sequential	Búsqueda insertado	<code>Customer_Id = 'SEQHASH002'</code>	15	0	1.612
Customers 100000	B+ Tree	Inserción	Nuevo cliente con fecha 2021-06-15	23	2	3.133
Customers 100000	B+ Tree	Búsqueda rango	<code>Subscription_Date BETWEEN '2021-01-01' AND '2021-12-31'</code>	124763	0	2323.093
Airbnb	Hash	Inserción	Nuevo <code>id</code>	43	4	9.180
Airbnb	Hash	Búsqueda	<code>id = 5881122</code>	16	0	4.690
Airbnb	B+ Tree	Inserción	Nuevo <code>price</code>	38	3	11.420
Airbnb	B+ Tree	Búsqueda rango	<code>price BETWEEN 100 AND 2000</code>	54174	0	627.540
Airbnb	R-Tree	Inserción	Nuevo <code>coords</code>	46	4	10.590
Airbnb	R-Tree	Radius Search	<code>coords IN (POINT(...), RADIUS 1)</code>	16813	0	188.980
Airbnb	R-Tree	kNN	<code>coords IN (POINT(...), K 10)</code>	442	0	4.370

6.4. Discusión y Análisis de Resultados

Los resultados experimentales muestran que las búsquedas exactas con índices orientados a igualdad son las más eficientes en accesos a disco y tiempo. En las pruebas de *Hash*, las búsquedas por identificador en `pokemon_complete.2025`, `customers_10000` y `airbnb_locations` se mantuvieron con un número bajo de lecturas, porque el índice permite ubicar directamente el bucket o entrada donde se encuentra el offset del registro. Esto confirma que el hashing extensible es adecuado para consultas puntuales, aunque no conserva orden y por ello no es apropiado para rangos.

El *Sequential File* también obtuvo buenos resultados en búsquedas exactas cuando el índice se encuentra ordenado o después de una reconstrucción. En `customers_10000`, la búsqueda post-rebuild sobre B050 tuvo menos lecturas que la búsqueda regular sobre un valor original del dataset, lo que evidencia el beneficio de reorganizar físicamente el área principal y vaciar el área auxiliar. Sin embargo, la inserción acumulada en esta técnica puede generar costo adicional cuando se dispara la reconstrucción, porque debe fusionar la zona ordenada con el área de desbordamiento.

Las consultas de rango con B+ Tree fueron las más costosas en lecturas cuando el rango seleccionado devuelve muchos registros. Esto se observa especialmente en `customers_100000`, donde el rango de fechas produjo 124763 lecturas y 2323.093 ms. El resultado es esperable: el B+ Tree reduce el costo de ubicar el inicio del rango, pero luego debe recorrer hojas y recuperar muchos registros desde el heap file. Por tanto, su eficiencia depende tanto del índice como de la selectividad del rango consultado.

En las pruebas espaciales, el R-Tree mostró una diferencia clara entre *kNN* y *Radius Search*. La consulta kNN tuvo 442 lecturas y 4.370 ms, porque el algoritmo puede priorizar los nodos cercanos y detenerse al encontrar los vecinos requeridos. En cambio, el radio de búsqueda produjo 16813 lecturas y 188.980 ms, debido a que un radio amplio puede intersectar muchos MBR y obligar al índice a explorar varias ramas del árbol.

En general, las inserciones mantienen un número moderado de escrituras porque el sistema escribe el registro en el heap file y luego actualiza los archivos de índice asociados. Las diferencias entre técnicas aparecen cuando la estructura necesita mantenimiento adicional: split de buckets en Hash, rebuild en Sequential File, split/propagación en B+ Tree o ajuste de MBR en R-Tree. Esto demuestra que el costo de inserción no depende únicamente del tamaño del registro, sino también del estado interno de la estructura de indexación.

7. Conclusiones

El proyecto implementó un simulador de gestor de base de datos desde cero, sin apoyarse en motores externos, integrando almacenamiento en heap file, acceso paginado a disco, parser SQL, motor de ejecución, backend HTTP y frontend de consulta. La arquitectura separa correctamente los registros reales de las estructuras de índice: el heap file almacena las tuplas completas y cada índice mantiene claves con offsets físicos hacia dichos registros.

Durante el desarrollo se implementaron y probaron cuatro técnicas de indexación con propósitos distintos: Sequential File para búsquedas ordenadas y rangos, Extendible Hashing para búsquedas exactas, B+ Tree para rangos generales y R-Tree para consultas espaciales. Las pruebas experimentales permitieron observar que cada técnica tiene ventajas según el tipo de consulta: Hash destaca en igualdad, B+ Tree en rangos selectivos, Sequential File mejora tras rebuild y R-Tree permite búsquedas espaciales que serían costosas con escaneo secuencial.

También se comprobó la importancia de medir accesos a disco además del tiempo de ejecución. Dos consultas pueden parecer similares desde SQL, pero producir costos muy distintos según la cantidad de páginas recorridas, la selectividad del predicado y el mantenimiento interno del índice. Esta medición permitió relacionar directamente la teoría de organización de archivos

con el comportamiento real del sistema.

Como trabajo futuro, se podría mejorar el proyecto incorporando manejo más completo de SQL, soporte para inserciones parciales, optimización de consultas por rango sobre data-sets grandes, visualización más detallada de R-Tree y herramientas automáticas para ejecutar benchmarks completos. Asimismo, sería útil reforzar las pruebas de estrés para splits, rebuilds y borrados, de manera que el sistema pueda evaluarse con mayor precisión frente a cargas de trabajo más grandes y variadas.

8. Anexos

8.1. Repositorio del proyecto

El código fuente del proyecto se encuentra disponible en el siguiente repositorio:

- Repositorio en GitHub

8.2. Video de demostración

El video de demostración y material complementario se encuentra disponible en el siguiente enlace:

- Carpeta de Google Drive

8.3. Evidencia de ejecución

En esta sección se incluirán capturas de pantalla del funcionamiento del frontend, backend, ejecución de consultas y resultados experimentales.

The screenshot shows the DB_ENGINE_EXPLORER_V3 interface. At the top, there's a status bar with 'DISK_R: 23', 'DISK_W: 2', and 'MS: 47.22'. Below it, a code editor displays an SQL INSERT statement into the 'pokemon_complete_2025' table. The statement is successful, as indicated by the 'Operation Successful' message. To the right, a 'SCHEMA CATALOG' panel shows the 'pokemon_complete_2025' table with 1028 records. Below the schema, a 'RAW ENGINE RESPONSE' panel displays the JSON response for the inserted record, including fields like 'pokemon_id', 'name', 'genus', 'generation', 'type_1', 'type_2', 'base_stats', 'height', 'weight', 'base_experience', 'ability_1', 'hidden_ability', 'color', 'shape', 'habitat', 'growth_rate', 'egg_groups', 'is_legendary', and 'is_mythical'.

```

INSERT INTO pokemon_complete_2025 VALUES (10001, 'btreemon', 'Test Pokemon', 'X', 'normal', '', 1, 70, 80, 75, 80, 75, 90, 470, 1.2, 20.0, 80, 'test ability', '', '', 'gray', 'unknown', 'unknown', 'medium', 'monster', 'False', 'False', 'False', 45, 70, 20, 1, 'Pokemon de prueba btree.', 'https://example.com/btreemon.png', 'False', 13.0, 1.00, 155, 155, 160, 150, 'unknown', 'Average');

Idle.
Op: INSERT

Operation Successful

{
  "operation": "insert",
  "table": "pokemon_complete_2025",
  "db_offset": 1123730,
  "row": {
    "pokemon_id": 10001,
    "name": "btreemon",
    "genus": "Test Pokemon",
    "generation": "X",
    "type_1": "normal",
    "type_2": "",
    "base_stats": {
      "hp": 70,
      "attack": 80,
      "defense": 75,
      "sp_attack": 80,
      "sp_defense": 75,
      "speed": 90,
      "base_stat_total": 470,
      "height_m": 1.2,
      "weight_kg": 20,
      "base_experience": 80,
      "ability_1": "test ability",
      "ability_2": "",
      "hidden_ability": "",
      "color": "gray",
      "shape": "unknown",
      "habitat": "unknown",
      "growth_rate": "medium",
      "egg_groups": "monster",
      "is_legendary": "false",
      "is_mythical": "false"
    }
  }
}

> Catalog updated: 1 tables.
> SQL: SELECT * FROM pokemon_complete_2025 WHERE name = "pikachu";
> Execution finished in 31.00ms (Total).
> Catalog updated: 1 tables.
> SQL: INSERT INTO pokemon_complete_2025 VALUES (10001, 'btreemon', 'Test Pokemon', 'X', 'normal', '', 1, 70, 80, 75, 80, 75, 90, 470, 1.2, 20.0, 80, 'test ability', '', '', 'gray', 'unknown', 'unknown', 'medium', 'monster', 'False', 'False', 'False', 45, 70, 20, 1, 'Pokemon de prueba btree.', 'https://example.com/btreemon.png', 'False', 13.0, 1.00, 155, 155, 160, 150, 'unknown', 'Average');
> Execution finished in 51.00ms (Total).
> Catalog updated: 1 tables.
  
```

Figura 2: Evidencia de ejecución 1

DB_ENGINE_EXPLORER_V3

DISK_R: 69 DISK_W: 7 MS: 6.04

```
INSERT INTO customers_10000 VALUES (210001, '0001', 'Seqtest001', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq001@test.com', '2026-05-09', 'https://seq001.local');
INSERT INTO customers_10000 VALUES (210002, '0002', 'Seqtest002', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq002@test.com', '2026-05-09', 'https://seq002.local');
INSERT INTO customers_10000 VALUES (210003, '0003', 'Seqtest003', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq003@test.com', '2026-05-09', 'https://seq003.local');
```

Idle.

EJECUTAR [CTRL+ENTER]

Op: INSERT

Operation Successful

```
{
  "operation": "insert",
  "table": "customers_10000",
  "db_offset": 800000,
  "row": [
    {
      "id": 210001,
      "customer_id": "0001",
      "first_name": "Seqtest001",
      "last_name": "Overflow",
      "company": "UTEC",
      "city": "Lima",
      "country": "Peru",
      "phone_1": "000",
      "phone_2": "000",
      "email": "seq001@test.com",
      "subscription_date": "2026-05-09",
      "website": "https://seq001.local"
    }
  ]
}
```

```
{
  "operation": "insert",
  "table": "customers_10000",
  "db_offset": 800000,
  "row": [
    {
      "id": 210002,
      "customer_id": "0002",
      "first_name": "Seqtest002",
      "last_name": "Overflow",
      "company": "UTEC",
      "city": "Lima",
      "country": "Peru",
      "phone_1": "000",
      "phone_2": "000",
      "email": "seq002@test.com",
      "subscription_date": "2026-05-09",
      "website": "https://seq002.local"
    }
  ]
}
```

```
{
  "operation": "insert",
  "table": "customers_10000",
  "db_offset": 800000,
  "row": [
    {
      "id": 210003,
      "customer_id": "0003",
      "first_name": "Seqtest003",
      "last_name": "Overflow",
      "company": "UTEC",
      "city": "Lima",
      "country": "Peru",
      "phone_1": "000",
      "phone_2": "000",
      "email": "seq003@test.com",
      "subscription_date": "2026-05-09",
      "website": "https://seq003.local"
    }
  ]
}
```

```
{
  "stats": {
    "execution_ms": 6.035,
    "disk_read": 69,
    "disk_write": 7
  }
}
```

> SQL: SELECT * FROM customers_10000 WHERE id = 1;

> Execution finished in 11.00ms (total).

> Catalog updated: 3 tables.

> SQL: INSERT INTO customers_10000 VALUES (210001, '0001', 'Seqtest001', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq001@test.com', '2026-05-09', 'https://seq001.local');

> Execution finished in 11.00ms (total).

> SQL: INSERT INTO customers_10000 VALUES (210002, '0002', 'Seqtest002', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq002@test.com', '2026-05-09', 'https://seq002.local');

> Execution finished in 11.00ms (total).

> SQL: INSERT INTO customers_10000 VALUES (210003, '0003', 'Seqtest003', 'Overflow', 'UTEC', 'Lima', 'Peru', '000', '000', 'seq003@test.com', '2026-05-09', 'https://seq003.local');

> Execution finished in 11.00ms (total).

> Catalog updated: 3 tables.

Figura 3: Evidencia de ejecución 2

DB_ENGINE_EXPLORER_V3

DISK_R: 23 DISK_W: 2 MS: 4.23

```
INSERT INTO customers_10000 VALUES (300001, 'HASH0001', 'Sebastian', 'HashTest', 'UTEC', 'Lima', 'Peru', '000', '000', 'hash001@test.com', '2026-05-09', 'https://hash001.local');
```

Idle.

EJECUTAR [CTRL+ENTER]

Op: INSERT

Operation Successful

```
{
  "operation": "insert",
  "table": "customers_10000",
  "db_offset": 300000,
  "row": [
    {
      "id": 300001,
      "customer_id": "HASH0001",
      "first_name": "Sebastian",
      "last_name": "HashTest",
      "company": "UTEC",
      "city": "Lima",
      "country": "Peru",
      "phone_1": "000",
      "phone_2": "000",
      "email": "hash001@test.com",
      "subscription_date": "2026-05-09",
      "website": "https://hash001.local"
    }
  ]
}
```

```
{
  "stats": {
    "execution_ms": 4.235,
    "disk_read": 23,
    "disk_write": 2
  }
}
```

> System initialized. Waiting for input.

> Catalog updated: 3 tables.

> SQL: SELECT * FROM customers_10000 WHERE subscription_date BETWEEN '2026-01-01' AND '2026-12-31';

> Execution finished in 359.20ms (total).

> Catalog updated: 3 tables.

> SQL: INSERT INTO customers_10000 VALUES (300001, 'HASH0001', 'Sebastian', 'HashTest', 'UTEC', 'Lima', 'Peru', '000', '000', 'hash001@test.com', '2026-05-09', 'https://hash001.local');

> Execution finished in 4.24ms (total).

> Catalog updated: 3 tables.

Figura 4: Evidencia de ejecución 3

DB_ENGINE_EXPLORER_V3 DISK_R: 124761 DISK_W: 0 MS: 2323.00

```
SELECT * FROM customers_100000 WHERE Subscription_Date BETWEEN '2021-01-01' AND '2021-12-03';
```

Idle. EJECUTAR [CTRL+ENTER]

Op: SELECT | Rows: 15220

id	customer_id	first_name	last_name	company	city	country	phone_1	phone_2
40208	4B39A97F7FA2	Bridget	Minters	Hutchinson Inc	South Colton	Paraguay	+1-954-047-8054	438-444-4846
40211	A9538410CF81C7	Nicole	Sandoval	Perkins-Shannon	Brendaport	Maldives	8287604682	(295)887-3638
40214	3cfc3580664387	Dale	Calhoun	Stanton-Kemp	East Devintown	Benin	313-403-6427x115	8253266269
40215	620a37a8bda9fc	Gregory	Gonzales	Buchanan-Barrera	East Perry	Uganda	001-558-139-1744	001-938-258-2826x302
40217	8ed719cc33d89f	Karlina	Hayes	Bond LLC	North Rachel	Northern Mariana Islands	001-338-548-5879x8862	001-013-397-6831
40219	08001596288Ede	Brent	Duran	Skinner LLC	Jayborough	Equatorial Guinea	818-636-9347x2081	+1-269-065-4722x54001
40222	5CF34cE0a950aA	Brady	Fuentes	Pittman-Walter	North Rileymouth	Nigeria	533-583-1836x3358	688-638-6496x17471
40225	EC0N8dxZx2a52	Marvin	Comacho	Mann-Lang	Port Jacksonville	Georgia	456-229-5388x21922	200-082-2449
330001	87H6S4001	SteveInsert	Customer	UTEC	Lima	Peru	000	000

> Function finished in 6.38ms (Total).
 > Catalog updated: 1 tables.
 > SQL: INSERT INTO customers_100000 VALUES (330001, '87H6S4001', 'SteveInsert', 'Customer', 'UTEC', 'Lima', 'Peru', '000', '000', 'sthash001@test.com', '2021-06-15', 'https://sthash001.local');
 > Function finished in 8.0ms (Total).
 > Catalog updated: 1 tables.
 > SQL: SELECT * FROM customers_100000 WHERE Subscription_Date BETWEEN '2021-01-01' AND '2021-12-03';
 > Function finished in 2372.26ms (Total).
 > Catalog updated: 1 tables.

Figura 5: Evidencia de ejecución 4

DB_ENGINE_EXPLORER_V3 DISK_R: 412 DISK_W: 0 MS: 4.37

```
SELECT * FROM airbnb_locations
WHERE coords IN (
  POINT(40.7580, -73.9855),
  K 50
);
```

Idle. EJECUTAR [CTRL+ENTER]

Op: SELECT | Rows: 50

id	name	host_id	host_name	neighbourhood_group	neighbourhood	coords
2165933	Furnished one bedroom in Midtown West.	11051885	Gregory	Manhattan	Theater District	[40.76251983642578, 3.98375701904297]
564447	GREAT DEAL IN TIMES SQ./HK	2775674	Matt	Manhattan	Hell s Kitchen	[40.75767135620117, 3.99034118652344]
2185842	1BR Superbowl rental Hell's Kitchen	6918298	Anna	Manhattan	Hell s Kitchen	[40.7599983215332, -9.9908894165039]
5104668	Alcove Studio 1 blk from Times Sq	24915685	Naveen	Manhattan	Hell s Kitchen	[40.759281158447266, 73.9903564453125]

> Catalog updated: 1 tables.
 > SQL: SELECT * FROM airbnb_locations WHERE coords IN (POINT(40.7580, -73.9855), K 50);
 > Function finished in 23.00ms (Total).
 > Catalog updated: 1 tables.

SCHEMA CATALOG
airbnb_locations (10998 recs)

RAW ENGINE RESPONSE

```
{
  "ok": true,
  "results": [
    {
      "operation": "select",
      "table": "airbnb_locations",
      "count": 50,
      "rows": [
        {
          "id": 6328973,
          "name": "Times Square room with private bath",
          "host_id": "31626211",
          "host_name": "Troy."
        }
      ]
    }
  ]
}
```

Figura 6: Evidencia de ejecución 5

DB_ENGINE_EXPLORER_V3 DISK_R: 412 DISK_W: 0 MS: 4.37

```
SELECT * FROM airbnb_locations
WHERE coords IN (
  POINT(40.7580, -73.9855),
  K 50
);
```

Idle. EJECUTAR [CTRL+ENTER]

Op: SELECT | Rows: 50

id	name	host_id	host_name	neighbourhood_group	neighbourhood	coords
2165933	Furnished one bedroom in Midtown West.	11051885	Gregory	Manhattan	Theater District	[40.76251983642578, 3.98375701904297]
564447	GREAT DEAL IN TIMES SQ./HK	2775674	Matt	Manhattan	Hell s Kitchen	[40.75767135620117, 3.99034118652344]
2185842	1BR Superbowl rental Hell's Kitchen	6918298	Anna	Manhattan	Hell s Kitchen	[40.7599983215332, -9.9908894165039]
5104668	Alcove Studio 1 blk from Times Sq	24915685	Naveen	Manhattan	Hell s Kitchen	[40.759281158447266, 73.9903564453125]

> Catalog updated: 1 tables.
 > SQL: SELECT * FROM airbnb_locations WHERE coords IN (POINT(40.7580, -73.9855), K 50);
 > Function finished in 23.00ms (Total).
 > Catalog updated: 1 tables.

SCHEMA CATALOG
airbnb_locations (10998 recs)

RAW ENGINE RESPONSE

```
{
  "ok": true,
  "results": [
    {
      "operation": "select",
      "table": "airbnb_locations",
      "count": 50,
      "rows": [
        {
          "id": 6328973,
          "name": "Times Square room with private bath",
          "host_id": "31626211",
          "host_name": "Troy."
        }
      ]
    }
  ]
}
```

Figura 7: Evidencia de ejecución 6